

Universidade de Brasília (UnB)
Disciplina: Teleinformática e Redes 1 (TR1)
Professor: Marcelo Antonio Marotta

Simulador TR1

Camadas Física e de Enlace

Trabalho Final



Integrantes do Grupo:

Gustavo Henrique Andrade Cavalcanti
Fernando Augusto Hortencio

Brasília — 2026

1 Introdução

O **Simulador TR1** simula a transmissao de uma mensagem de texto entre dois nos de uma rede, de forma didatica e fiel. Ele foi desenvolvido na disciplina de Teleinformatica e Redes 1 (TR1).

O foco e exercitar, de ponta a ponta, as camadas fisica e de enlace do modelo de protocolos. Isso abrange o enquadramento dos dados, a deteccao e a correcao de erros, a modulacao em banda-base e a modulacao por portadora. A mensagem atravessa um meio de comunicacao ruidoso, modelado por amostras em Volts somadas a ruido gaussiano.

Para o usuario, o uso e direto. Ele digita um texto e escolhe, na interface grafica, os protocolos de cada camada e os parametros de ruido do canal.

A partir dessas escolhas, o simulador converte o texto em bits (camada de aplicacao), monta os quadros com os codigos de deteccao, o Hamming e o enquadramento (camada de enlace) e transforma os bits em sinal eletrico, com modulacao banda-base ou por portadora (camada fisica). O sinal propaga pelo meio, que soma ruido gaussiano, e percorre o caminho inverso no receptor.

O resultado aparece em duas abas: Transmissor e Receptor. Cada aba mostra os bits e sinais de cada camada, o texto recuperado e metricas como potencias, relacao sinal-ruido (SNR) e contagem de erros.

Um diferencial exigido pelo enunciado e a separacao explicita entre transmissor e receptor. Os dois executam em *threads* independentes, que se comunicam apenas por filas (`queue.Queue`). As filas fazem o papel do meio fisico.

O ruido e somado justamente no canal, entre as duas *threads*. Assim, o meio fica como o unico ponto de contato entre transmissor e receptor.

2 Arquitetura do Simulador

O simulador foi escrito em Python 3. A interface grafica usa PyGObject (GTK 3), e os graficos dos sinais sao desenhados no proprio GTK com `Gtk.DrawingArea`. Assim, os protocolos e a visualizacao principal nao dependem de bibliotecas externas de calculo ou plotagem.

O enunciado exige os modulos `CamadaFisica`, `CamadaEnlace`, `InterfaceGUI` e `Simulador`. O projeto entrega esses modulos em *snake_case* (padrao da linguagem) e acrescenta modulos auxiliares para a camada de aplicacao, o meio de comunicacao e a suite de testes.

A Tabela 1 resume o papel de cada arquivo e o requisito que ele atende.

Tabela 1: Mapeamento de arquivos do projeto e requisitos do enunciado.

Arquivo	Papel	Requisito
<code>simulador.py</code>	Rotina principal: orquestra TX $\rightarrow$ meio $\rightarrow$ RX em <i>threads</i>	Simulador
<code>camada_fisica.py</code>	Modulação digital (banda-base) e por portadora	CamadaFisica
<code>camada_enlace.py</code>	Enquadramento, detecção e correção de erros	CamadaEnlace
<code>interface_gui.py</code>	GUI em GTK 3 (entrada, configuração, gráficos)	InterfaceGUI
<code>camada_aplicacao.py</code>	Texto $\leftrightarrow$ bits (codificador/decodificador)	apoio
<code>meio_comunicacao.py</code>	Canal: sinal em Volts + ruído gaussiano $n(x, \sigma)$	apoio (meio)
<code>testes.py</code>	Suíte de testes de ida-e-volta (TX $\rightarrow$ RX sem ruído == entrada)	validação

2.1 Diagrama de Pilha (TX $\rightarrow$ MEIO $\rightarrow$ RX)

No transmissor, o texto desce pelas camadas de aplicacao, enlace e fisica ate virar um sinal eletrico. No meio, o ruido gaussiano e somado amostra a amostra. No receptor, o sinal sobe de volta pelas camadas ate ser reconstruido como texto. A comunicacao entre as *threads* ocorre por duas filas: TX $\rightarrow$ [canal soma ruído] $\rightarrow$ RX.

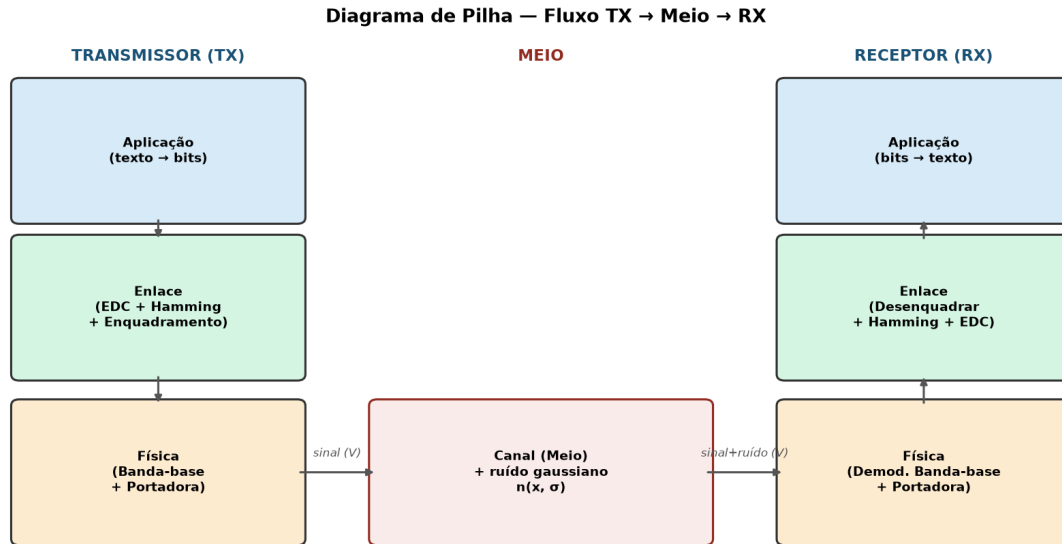


Figura 1: Diagrama de pilha TX $\rightarrow$ Meio $\rightarrow$ RX, com as camadas Aplicação, Enlace e Física no transmissor e no receptor e o canal somando ruído gaussiano.

3 Implementação

Esta secão detalha cada camada: o funcionamento dos protocolos e as decisoes de projeto.

Os algoritmos centrais (CRC, Hamming, checksum e todas as modulacoes) foram implementados manualmente, bit a bit, sem bibliotecas externas como a `zlib`, conforme exige o enunciado.

Os parametros globais da camada fisica sao: $V = 1,0$ V de amplitude, 100 amostras por bit, 100 amostras por simbolo, 4 ciclos de portadora por simbolo e o par de frequencias (2, 4) ciclos para o FSK.

3.1 Camada Física — Modulação Digital (Banda-Base)

Foram implementadas tres codificacoes de linha.

No **NRZ-Polar**, o bit 1 vira $+V$ e o bit 0 vira $-V$. No receptor, a decisao usa a media das amostras de cada bit.

No **Manchester**, adota-se a convencao de Tanenbaum/G.E. Thomas: XOR com o clock. O bit 0 e uma transicao de baixo para alto e o bit 1, de alto para baixo. A decisao compara a media da primeira metade do periodo com a da segunda.

No **Bipolar (AMI)**, o bit 0 corresponde a 0 V e o bit 1 alterna entre $+V$ e $-V$, o que elimina a componente DC. A decisao usa o criterio $|media| > V/2$.

As funcoes seguem o padrao `modular_x / demodular_x` para cada esquema.

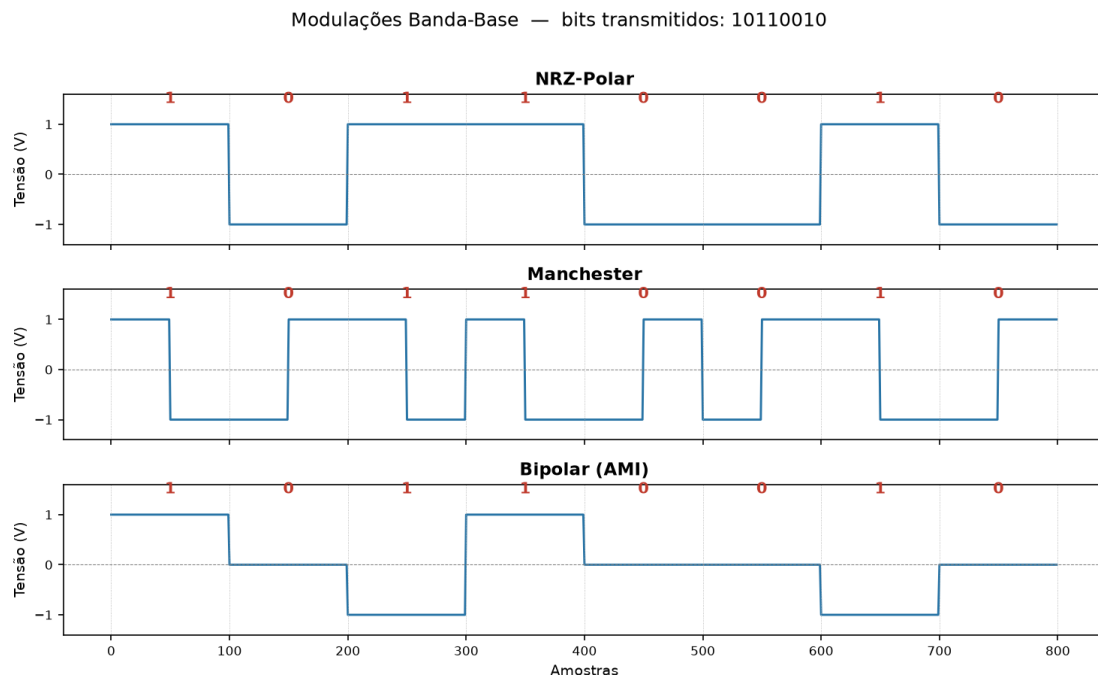


Figura 2: Formas de onda das modulações banda-base (NRZ-Polar, Manchester e Bipolar) para a sequência de bits 10110010.

3.2 Camada Física — Modulação por Portadora

As modulações por portadora usam uma representação comum em I/Q, na forma $s(t) = I \cos(2\pi ft) - Q \sin(2\pi ft)$. A demodulação é coerente, por correlação (produto interno), e escolhe o ponto da constelação mais próximo.

No **ASK** (on-off keying), o bit 1 é transmitido com a portadora em amplitude V e o bit 0 com $0 V$; a decisão compara a amplitude medida com $V/2$.

No **FSK**, usam-se duas frequências ortogonais (2 e 4 ciclos por símbolo) e a decisão olha em qual frequência há mais energia.

O **QPSK** transporta 2 bits por símbolo com mapeamento Gray. O **16-QAM** transporta 4 bits por símbolo, com níveis Gray $\{-3, -1, +1, +3\} \cdot (V/3)$ em cada eixo I e Q, decididos de forma independente.

Funções auxiliares como `onda`, `correlacionar`, `pad` e `bits_do_ponto_mais_proximo` são reaproveitadas entre os esquemas.

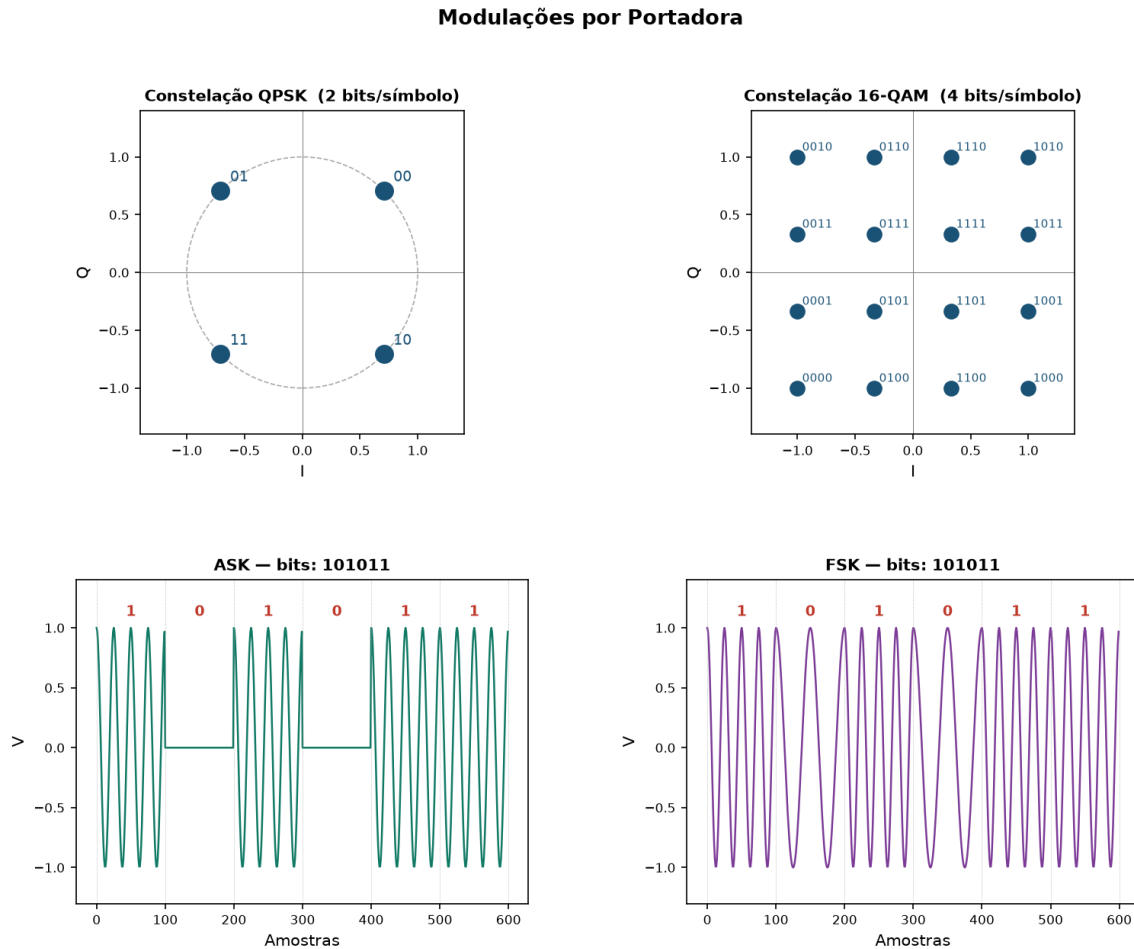


Figura 3: Constelações QPSK (2 bits/símbolo) e 16-QAM (4 bits/símbolo) com mapeamento Gray, e formas de onda ASK e FSK para a sequência 101011.

A Figura 4 ilustra como o ruído desloca os pontos da constelação QPSK, tornando a decisão de mínima distância progressivamente mais suscetível a erros.

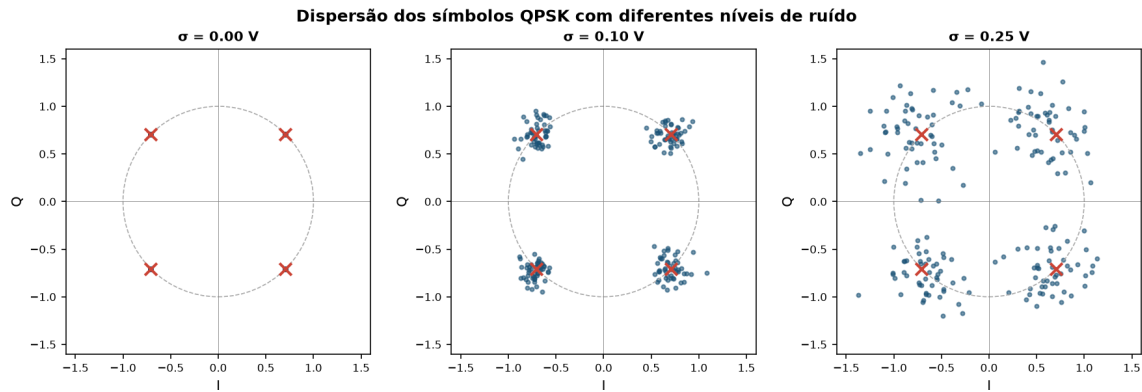


Figura 4: Dispersão dos símbolos QPSK recebidos para três níveis de ruído ($\sigma = 0,00$, $0,10$ e $0,25$ V). A cruz vermelha marca o ponto ideal da constelação; os pontos azuis são os símbolos recebidos após o canal ruidoso.

3.3 Camada de Enlace — Enquadramento

Foram implementadas tres tecnicas de enquadramento.

Na **contagem de caracteres**, um byte de cabecalho indica o numero de bytes do payload, limitado a no maximo 255.

No **byte stuffing** (FLAGS com insercao de bytes), adota-se `FLAG = 0x7E` e `ESC = 0x7D`. Quando esses padroes aparecem nos dados, um byte ESC e inserido antes deles.

No **bit stuffing**, a FLAG e `01111110`. Apos cinco bits 1 consecutivos no payload, insere-se um bit 0 para garantir que a FLAG nunca apareca nos dados.

As funcoes seguem o padrao `enquadrar_x` / `desenquadrar_x`.

3.4 Camada de Enlace — Detecção de Erros

Foram desenvolvidos tres mecanismos de deteccao de erros, todos sem bibliotecas externas.

O **bit de paridade par** anexa um byte (sete zeros mais o bit de paridade) para manter o alinhamento em bytes, e detecta um numero impar de bits invertidos.

O **checksum** usa a soma em complemento de 1 de palavras de 16 bits, com *end-around carry*, exatamente como o checksum da Internet.

O **CRC-32** (IEEE 802) usa o polinomio `0x04C11DB7` na forma refletida `0xEDB88320`, com inicializacao `0xFFFFFFFF` e XOR final. Sua corretude foi validada contra o vetor de teste conhecido: `CRC32("123456789") = 0xCBF43926`.

As funcoes seguem o padrao `adicionar_x` / `verificar_x`.

3.5 Camada de Enlace — Correção de Erros (Hamming)

A correcao de erros usa o codigo de **Hamming(8,4) estendido (SECDED)**, que transforma cada bloco de 4 bits em 8 bits. Ele corrige um erro e detecta dois erros por bloco.

A decodificacao combina a síndrome com o bit de paridade geral p_0 . Se a síndrome e não nula e a paridade acusa erro, ha um erro unico corrigivel. Se a síndrome e não nula mas a paridade esta correta, detecta-se um erro duplo, que não da para corrigir.

A escolha do Hamming estendido, em vez do (7,4) puro, foi deliberada: além de detectar o erro duplo, ele mantém o alinhamento em bytes, ao dobrar 1 byte em 2 bytes.

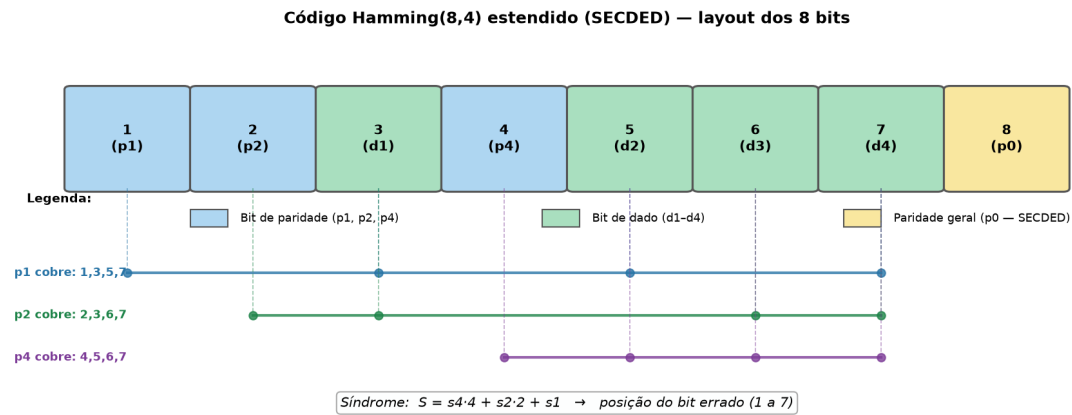


Figura 5: Layout do bloco Hamming(8,4) estendido: posições dos bits de paridade (p_1, p_2, p_4, p_0) e de dado (d_1-d_4), coberturas de cada bit de paridade e fórmula da síndrome.

3.6 Interface Gráfica (GTK 3)

A interface foi feita em GTK 3 (não em terminal), conforme exigido, pela classe `JanelaSimulador` em `interface_gui.py`.

O painel esquerdo reúne a configuração: o texto de entrada, o tamanho máximo de quadro, os menus de enquadramento, detecção, correção, modulação digital e modulação por portadora, e os parâmetros de ruído x e σ .

O painel direito mostra métricas gerais, uma tabela de processamento por fase e duas abas, Transmissor e Receptor. Cada aba mostra os bits de cada camada, o texto recuperado, um relatório por quadro (EDC OK ou com erro, bits corrigidos e erro duplo) e os gráficos dos sinais desenhados diretamente em GTK. A separação entre TX e RX fica explícita, atendendo ao enunciado.

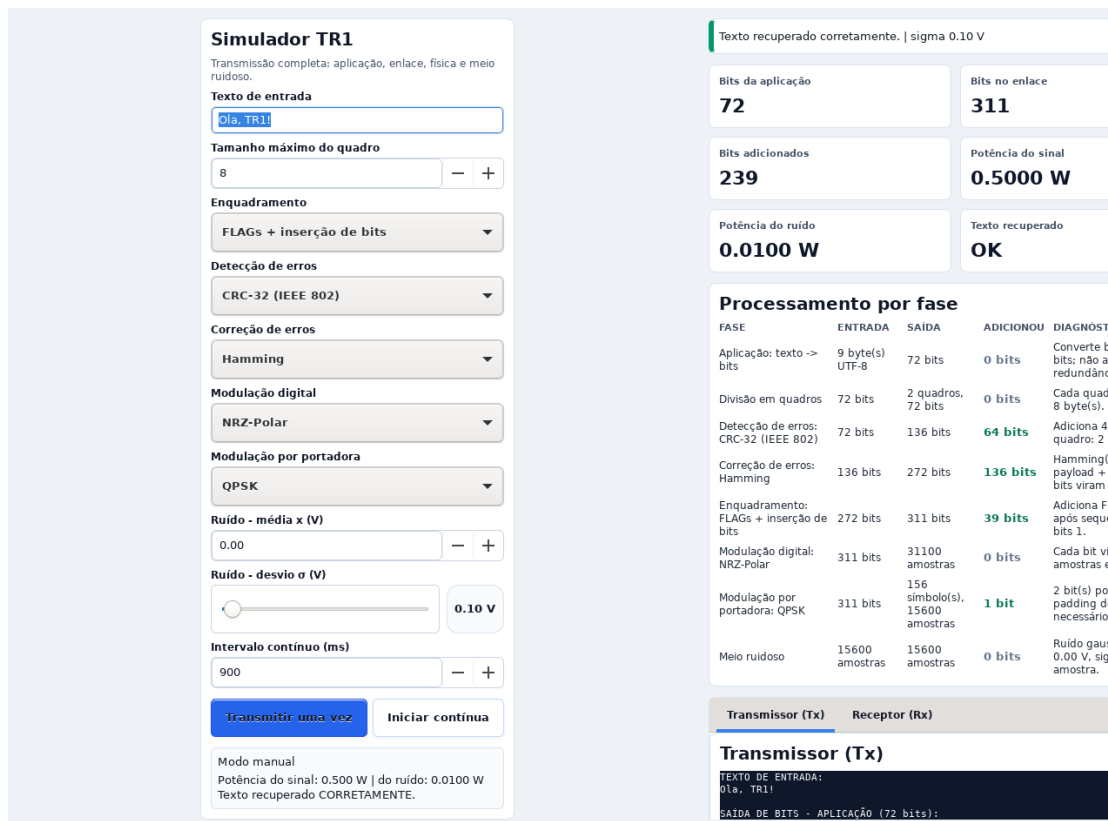


Figura 6: Interface gráfica em GTK 3: painel de configuração à esquerda, métricas no painel principal, tabela por fase e abas Transmissor/Receptor com gráficos nativos dos sinais.

3.7 Fluxo dos Pipelines de Transmissão e Recepção

No transmissor, `camada_enlace.transmitir(bits, config)` divide os bits em blocos de até `tam_max_quadro` bytes. A cada bloco anexa o código de detecção (paridade, checksum ou CRC); se o Hamming estiver ativo, o conjunto payload mais EDC é codificado, dobrando de tamanho; e o bloco é enquadrado. Os quadros são concatenados num único fluxo de bits. Em seguida, `simulador._rotina_tx` aplica a modulação digital banda-base e, se houver portadora, a modulação por portadora, gerando o sinal enviado ao meio.

No receptor, `simulador._rotina_rx` recebe o sinal ruidoso e demodula a portadora (ou a banda-base). Então `camada_enlace.receber` desenquadra, corrige com Hamming, verifica o EDC e entrega os bits a `camada_aplicacao.bits_para_texto`.

O meio, em `meio_comunicacao.transmitir(sinal, media, sigma)`, soma `random.gauss(media, sigma)` a cada amostra (ruído AWGN). Quando `media` e σ são zero, ele se comporta como canal ideal. A função `potencia_media` calcula P como a média de v^2 (em watts sobre 1Ω), usada no cálculo da SNR.

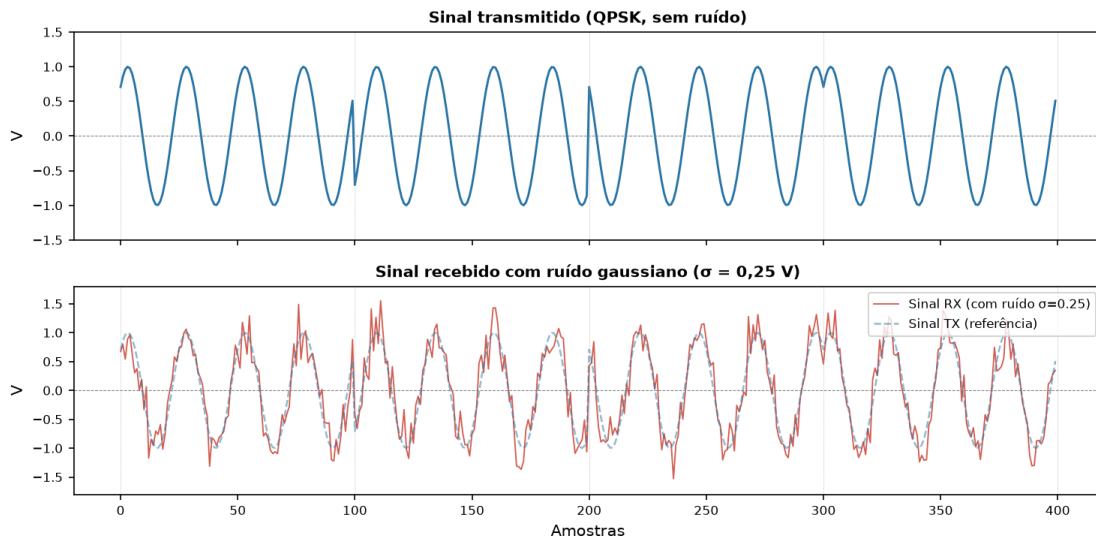


Figura 7: Sinal QPSK transmitido (limpo) e sinal recebido com ruído gaussiano ($\sigma = 0,25$ V). A linha tracejada é o sinal original de referência.

3.8 Decisões de Projeto e Casos Omissos

Alguns pontos não foram especificados no enunciado e exigiram decisões de projeto.

Todos os EDCs foram alinhados em bytes (paridade 1 byte, checksum 2 bytes e CRC 4 bytes), para combinar com os enquadramentos, que operam sobre bytes.

A ordem de aplicação é EDC primeiro e Hamming depois, para que o Hamming proteja também os bits de detecção. No receptor, a ordem se inverte: corrige antes de verificar o EDC.

No QPSK e no 16-QAM, o sinal é completado com zeros até fechar um símbolo inteiro (padding), e o desenquadramento descarta o excedente.

A convenção Manchester e a de Tanenbaum/G.E. Thomas, documentada de forma explícita. A demodulação foi feita robusta ao ruído: por média na banda-base e por correlação coerente com mínima distância na constelação por portadora, aproveitando o mapeamento Gray para reduzir bits errados.

A contagem de caracteres é limitada a 255 bytes por quadro, validada em `transmitir()` com um `ValueError` exibido na GUI. A decodificação de texto usa `errors="replace"`, para não interromper o programa quando o ruído corrompe bytes.

Por fim, o GTK é importado apenas no contexto da GUI, o que permite rodar os testes e os módulos em máquinas sem GTK.

4 Validação e Resultados

A corretude do simulador é verificada pela suite `testes.py`, com o critério de ida-e-volta: sem ruído, a saída do receptor deve ser idêntica à entrada do transmissor.

Os testes cobrem: a camada de aplicacao (texto para bits e de volta, incluindo acentos UTF-8); os tres enquadramentos, com seus casos criticos (payload so de bits 1 no bit stuffing e ocorrencia de FLAG/ESC no byte stuffing); os tres EDCs (aceitando quadros integros e detectando um bit invertido), com a conferencia do CRC-32 contra 0xCBF43926; o Hamming (ida-e-volta, correcao de um bit e deteccao de erro duplo); todas as modulacoes banda-base e por portadora, com e sem ruido; e a simulacao completa, combinando os 3 enquadramentos com as 5 opcoes de portadora.

O resultado atual da execucao e “**TODOS OS TESTES PASSARAM**”, o que evidencia a corretude da implementacao.

```
[PASS] aplicacao: texto -> bits -> texto
[PASS] enquadramento 'contagem': ida e volta
[PASS] enquadramento 'bytes': ida e volta
[PASS] enquadramento 'bits': ida e volta
[PASS] bit stuffing com payload só de 1s
[PASS] byte stuffing com FLAG/ESC no payload
[PASS] paridade: aceita payload íntegro
[PASS] paridade: detecta 1 bit invertido
[PASS] checksum: aceita payload íntegro
[PASS] checksum: detecta 1 bit invertido
[PASS] crc: aceita payload íntegro
[PASS] crc: detecta 1 bit invertido
[PASS] crc32 do vetor de teste '123456789' == 0xCBF43926
[PASS] hamming: ida e volta sem erro
[PASS] hamming: corrige 1 bit invertido
[PASS] hamming: detecta erro duplo
[PASS] banda-base 'nrz': ida e volta sem ruído
[PASS] banda-base 'nrz': ida e volta com ruído sigma=0.2
[PASS] banda-base 'manchester': ida e volta sem ruído
[PASS] banda-base 'manchester': ida e volta com ruído sigma=0.2
[PASS] banda-base 'bipolar': ida e volta sem ruído
[PASS] banda-base 'bipolar': ida e volta com ruído sigma=0.2
[PASS] portadora 'ask': ida e volta sem ruído
[PASS] portadora 'ask': ida e volta com ruído sigma=0.1
[PASS] portadora 'fsk': ida e volta sem ruído
[PASS] portadora 'fsk': ida e volta com ruído sigma=0.1
[PASS] portadora 'qpsk': ida e volta sem ruído
[PASS] portadora 'qpsk': ida e volta com ruído sigma=0.1
[PASS] portadora '16qam': ida e volta sem ruído
[PASS] portadora '16qam': ida e volta com ruído sigma=0.1
[PASS] simulação completa enq=contagem portadora=nenhuma
[PASS] simulação completa enq=contagem portadora=ask
[PASS] simulação completa enq=contagem portadora=fsk
[PASS] simulação completa enq=contagem portadora=qpsk
[PASS] simulação completa enq=contagem portadora=16qam
[PASS] simulação completa enq=bytes portadora=nenhuma
[PASS] simulação completa enq=bytes portadora=ask
[PASS] simulação completa enq=bytes portadora=fsk
[PASS] simulação completa enq=bytes portadora=qpsk
[PASS] simulação completa enq=bytes portadora=16qam
[PASS] simulação completa enq=bits portadora=nenhuma
[PASS] simulação completa enq=bits portadora=ask
[PASS] simulação completa enq=bits portadora=fsk
[PASS] simulação completa enq=bits portadora=qpsk
[PASS] simulação completa enq=bits portadora=16qam

RESULTADO GERAL: TODOS OS TESTES PASSARAM
```

Figura 8: Saída do terminal após execução de `testes.py`: todos os 45 testes passaram.

5 Membros e Atividades Desenvolvidas

O trabalho foi dividido entre os dois integrantes do grupo, mantendo uma separacao equilibrada entre protocolos, simulacao e interface.

Gustavo Henrique Andrade Cavalcanti ficou responsavel pela camada fisica e por parte da camada de enlace: modulacoes banda-base (NRZ-Polar, Manchester e Bipolar),

modulações por portadora (ASK, FSK, QPSK e 16-QAM), meio de comunicação com ruído gaussiano, enquadramento e mecanismos de detecção e correção de erros.

Fernando Augusto Hortencio ficou responsável pela integração do simulador, pela comunicação entre as *threads* de transmissor e receptor por filas, pela interface gráfica em GTK, pela interface web opcional, pela organização do repositório, pela documentação e pela validação final com testes de ida-e-volta.

Apesar dessa divisão, ambos revisaram os pontos centrais do projeto para dominar a explicação completa do código durante a apresentação.

6 Conclusão

O Simulador TR1 atingiu os objetivos propostos. Ele implementa todos os protocolos exigidos das camadas física e de enlace sem usar bibliotecas externas nos algoritmos centrais: CRC, Hamming, checksum e todas as modulações foram escritos manualmente, bit a bit.

A arquitetura baseada em *threads* de transmissor e receptor reproduz fielmente o diagrama do enunciado e isola o meio, com o ruído gaussiano, como o único ponto de contato entre as duas pontas.

As principais dificuldades foram: a demodulação robusta a ruído, que exigiu correlação coerente e critério de mínima distância na constelação; a manutenção do alinhamento em bytes entre EDC, Hamming e enquadramento; o tratamento do padding nas modulações multibit QPSK e 16-QAM; e a definição e documentação explícita da convenção de Manchester.

Superados esses pontos, a suíte de testes de ida-e-volta passou em todos os casos, confirmando a correção do sistema.

O resultado é uma ferramenta didática que torna tangíveis os conceitos das camadas física e de enlace, permitindo observar, na prática, o efeito do ruído, dos esquemas de modulação e dos mecanismos de detecção e correção de erros sobre a transmissão de dados.